

DoS Attack & Mitigation Lab

Technical Write-Up: Slowloris vs. nginx

CYB102 — Unit 4, Project 4

1. Objective

This lab demonstrates a low-bandwidth Denial-of-Service (DoS) attack and how to defend against it at the web-server layer. A Slowloris attack was launched against a local nginx server running inside a VM, the resulting connection exhaustion was observed in Netdata, mitigation rules were added to nginx, and the same attack was re-run to confirm the defense worked.

The core idea: Slowloris does not flood with traffic. It opens many connections and holds them open with deliberately slow, incomplete HTTP requests, exhausting the server's connection pool so legitimate users cannot connect. The defense is to make nginx *impatient* — close any connection that does not complete its request quickly, and cap how many connections a single IP may hold.

2. Environment

- **Target:** nginx web server, accessed at 127.0.0.1 (the VM's own loopback)
- **Attack tool:** slowloris-run
- **Monitoring:** Netdata, viewed in the VM's Firefox at <http://127.0.0.1:19999>
- **Access:** RDP into the VM (required so the browser and loopback target are on the same machine)

3. Setup & Verification

3.1 Confirm nginx is running

```
sudo systemctl status nginx
```

What it does: Queries systemd for the live state of the nginx service. A green `active` (running) line confirms the target is up.

When you use it: Any time you need to know whether a Linux service is alive before interacting with it. `systemctl status <service>` is the universal first diagnostic for managed services.

3.2 Confirm the attack tool is installed

```
which slowloris-run
```

What it does: `which` prints the full path of an executable found on your PATH, or nothing if it isn't installed. A path returned = installed and runnable.

3.3 Install Netdata

```
wget -O /tmp/netdata-kickstart.sh https://my-netdata.io/kickstart.sh \  
&& sudo sh /tmp/netdata-kickstart.sh
```

What it does: `wget -O` downloads the official installer script to a named file; the second half runs it. The kickstart script auto-detects the OS and installs Netdata as a service listening on port 19999.

Verify it's listening:

```
sudo systemctl status netdata
sudo ss -tlnp | grep 19999
```

`ss -tlnp` lists listening (-l) TCP (-t) sockets with numeric ports (-n) and owning processes (-p). Grepping for 19999 confirms Netdata is actually bound to the port before you open the browser. `ss` is the modern replacement for `netstat`.

3.4 Build the dashboard

In Netdata's Dashboard tab, surface charts for **Network Traffic** (`net.networks`) and **TCP Connections** (`ip.tcpsock`). The TCP Connections chart is the one that tells the story — it shows how many connections nginx is holding at any instant.

4. Attack 1 — Unprotected

```
slowloris-run 127.0.0.1 -s 500
```

What it does: Launches Slowloris against the loopback target with `-s 500` (500 sockets). Each socket opens a TCP connection and sends an incomplete HTTP request, then trickles out just enough data to avoid timing out — keeping the connection open and occupied.

What you observe: On the TCP Connections graph, the count climbs from a baseline of ~70 connections to **over 1,000** and holds there in flat-topped plateaus. With no timeout configured, nginx waits indefinitely for requests that never complete, so the pool stays saturated.

Let it run 30+ seconds, screenshot, then stop with `Ctrl+C`.

5. Attack 2 — Adding Mitigation

5.1 Back up the config first

```
sudo cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak
```

Why: Always snapshot a config before editing it. If a change breaks the service, you restore in one command. `cp source dest` is the standard copy; the `.bak` suffix is convention for a backup.

5.2 Add directives to the http {} block

Edit `/etc/nginx/nginx.conf` with `sudo nano /etc/nginx/nginx.conf` and add inside `http { }`:

```
limit_conn_zone $binary_remote_addr zone=addr:10m;

client_header_timeout 5s;
client_body_timeout   5s;
send_timeout          5s;
keepalive_timeout     5; # edit the EXISTING line, don't add a 2nd
```

What each directive does:

Directive	Purpose
<code>limit_conn_zone</code>	Declares a shared memory zone (10MB) keyed by client IP. This is the storage that <code>limit_conn</code> uses to count per-IP connections.
<code>client_header_timeout</code>	Drops a connection if the full request header isn't received within 5s. This is the primary Slowloris killer — Slowloris depends on sending headers slowly.

<code>client_body_timeout</code>	Same idea for the request body: incomplete body within 5s = connection closed.
<code>send_timeout</code>	Closes the connection if the client stops reading the response. Prevents slow-read variants from holding connections.
<code>keepalive_timeout</code>	How long an idle keep-alive connection stays open. Lowered from the default (65s) to 5s so idle sockets are released fast.

5.3 Add the connection cap to the server {} block

Edit `/etc/nginx/sites-available/default` and add inside `server { }`:

```
limit_conn addr 10;
```

What it does: Limits each client IP to 10 simultaneous connections, using the `addr` zone declared earlier. A single source can no longer open 500 connections — excess ones are rejected.

5.4 The duplicate `keepalive_timeout` error

nginx ships with a `keepalive_timeout` already present in the default `nginx.conf`. Adding a second triggers:

```
"keepalive_timeout" directive is duplicate in /etc/nginx/nginx.conf:68
```

Fix: Find both occurrences, delete the one you added, and edit the original's value instead.

```
grep -n keepalive_timeout /etc/nginx/nginx.conf
# delete the duplicate line you pasted, keep/lower the original
```

Lesson: Before adding a directive, check whether the default config already declares it. `grep -n <term> <file>` prints matching lines with line numbers — the fastest way to audit a config for existing settings.

5.5 Test and reload

```
sudo nginx -t && sudo systemctl reload nginx
```

What it does: `nginx -t` parses the config and reports syntax errors *without* applying them — it validates first. The `&&` means reload only runs if the test passes. `systemctl reload` applies the new config gracefully without dropping existing connections (unlike restart).

Remember this pattern: always `nginx -t && reload`. Testing before reloading prevents pushing a broken config to a live server.

6. Results — Unprotected vs. Protected

Re-running the identical attack (`slowloris-run 127.0.0.1 -s 500`) after mitigation produced a completely different graph. The key is the **shape**, not just the height:

- **Unprotected:** connections climb to ~1,000+ and hold in sustained, flat-topped plateaus. The pool stays saturated for the entire attack.
- **Protected:** connections spike briefly (sockets do open) but immediately collapse back to the ~50 baseline, producing a dense sawtooth. The 5-second timeouts force-close each slow connection before it can accumulate, so the pool is never saturated.

One-sentence summary: the difference is sustained saturation (vulnerable) versus connections that open and get force-closed before they can pile up (mitigated).

7. Key Takeaways Moving Forward

- **DoS is not always about volume.** Slowloris uses almost no bandwidth. The resource it exhausts is the connection pool, not the network pipe, so the defense has to match the type of attack.
- **Timeouts are a defense.** Generous default timeouts are an attack surface. Tightening header/body timeouts neutralizes slow-rate attacks directly.
- **Validate before applying.** `nginx -t && reload` should be muscle memory. Never reload an untested config on a live service.
- **Audit before editing.** `grep` the config for a directive before adding it, since duplicates are a common and avoidable error.
- **Read the axis, not just the line.** A graph in 'conns' vs 'K conns' tells very different stories. Confirm units and baseline before interpreting a spike as an attack.
- **Back up first.** A one-line `cp` before editing a config saves you from a broken service.